M16 - DataOps, Security & Production Operations

LEARNER BOOK - BEGINNER-FIRST TEACHING RC2

Primary teacher

The lesson explains from zero and gives a concrete case before asking for proof. Videos/docs are reinforcement, not a rescue requirement.

Contents

DE16-01 - Production mindset

What you will be able to do

Think in production invariants: repeatability, failure containment, auditability, ownership and rollback rather than "script ran once".

Why this matters

A data engineer owns behavior over time under failures, changes and other users, not only a correct local output.

Picture it this way

A prototype is a recipe tested once; production is a restaurant that must serve the same safe dish every day with logs, hygiene and emergency procedures.

Start from zero

- Production-ready is a spectrum with explicit risks/SLA, not a magic label.
- Define data contracts/grain/quality gates and failure behavior.
- Make runs reproducible from versioned code/config and observable with run IDs.
- Design idempotency/retry/backfill before incidents.
- Separate duties/environments/secrets and least privilege.
- Have ownership/runbook/rollback/recovery evidence.

Teacher demonstration / case

```
Prototype question: "Does it work?"
Production questions:
- Does rerun work?
- What fails safely?
- What data changed?
- Can we roll back/rebuild?
- Who is alerted/authorized?
- What evidence exists?
```

Read it like English

- The module is about operational discipline across earlier technical skills.

Expected check

property	proof
repeatable	same input/config -> same result
safe failure	prior good data/state protected
auditable	run/lineage/version metadata
recoverable	rollback/rebuild/runbook

Common beginner traps

- Calling tests alone production readiness.
- No owner/runbook because cloud UI has logs.
- Treating security as final checklist.

Now you do a similar task

Audit one earlier project against the six production questions and identify its top three gaps.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Which is stronger evidence: "no errors in last run" or a tested failure/rollback path?

Answer in your own words before continuing.

DE16-02 - Testing strategy for data engineering

What you will be able to do

Build a layered test strategy: pure/unit, data contract, integration, reconciliation and end-to-end smoke.

Why this matters

Different bugs require different tests; one giant end-to-end test is slow and gives poor diagnosis.

Picture it this way

Testing layers are safety nets at different heights: small nets catch local logic fast; larger nets catch system integration.

Start from zero

- Unit/pure transformation tests use tiny controlled inputs.
- Contract/schema tests validate required shape/types/keys.
- Integration tests verify database/file/API/container interfaces.
- Data quality/reconciliation tests validate population/measure invariants.
- End-to-end smoke validates a small realistic flow in an environment.
- Test failure should block the correct boundary and explain evidence.

Teacher demonstration / case

```
tests/
  test_transform.py      # pure logic
  test_contract.py       # schema/key
  test_reconciliation.py # row/measure controls
integration smoke        # controlled env
```

Read it like English

- The supplied project tests pipeline logic while quality-gate script tests data controls.

Expected check

test	catches
unit	logic regression
contract	schema/key drift
integration	interface/config issue
reconciliation	loss/fan-out
smoke	deployment/system wiring

Common beginner traps

- Only end-to-end manual testing.
- Only unit tests then assuming target data correct.
- Tests with production secrets/data.

Now you do a similar task

Use P01 test-strategy exercise on a pipeline and create a matrix of five failures vs the earliest test that should catch each.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why is a fast unit test not enough to prove a production database load is correct?

Answer in your own words before continuing.

DE16-03 - Automated quality gates

What you will be able to do

Turn quality rules into automated gates that change pipeline/CI/deploy status instead of passive reports.

Why this matters

A rule that logs ERROR but still publishes bad data is not a gate.

Picture it this way

A quality gate is the security door: when a blocking condition fails, shipment cannot pass.

Start from zero

- Define blocking invariants vs warnings with owner/SLA.
- Examples: required columns, key uniqueness, accepted domain, referential integrity, freshness, row/amount reconciliation.
- Run gate before publish/state advancement.
- Return non-zero/raise failure for blocking conditions in automation.
- Preserve rejected/diagnostic samples safely.
- Track gate metrics over time to tune thresholds without weakening integrity.

Teacher demonstration / case

```
python G02_QUALITY_GATE.py
# A blocking failure must change exit status / prevent publish.
# Inspect normal vs bad evidence.
```

Read it like English

- The supplied project includes normal/bad evidence for deterministic gate behavior.

Expected check

rule	typical gate
business key unique	BLOCK
source-target amount reconciliation	BLOCK
freshness near SLA threshold	WARN then BLOCK past SLA
optional field fill-rate	monitor/warn

Common beginner traps

- Logging warning and continuing for broken primary key.
- Thresholds chosen to make current data pass.
- Gate runs after serving publish.

Now you do a similar task

Run quality gate against normal and bad evidence and save exit/status difference.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

What makes a data-quality check a gate rather than just a metric?

Answer in your own words before continuing.

DE16-04 - Continuous Integration fundamentals

What you will be able to do

Explain CI as automated validation of a proposed code/config change before it is merged/deployed.

Why this matters

CI reduces the chance one developer's change silently breaks shared pipelines.

Picture it this way

Every code change goes through an automatic inspection lane before joining main.

Start from zero

- CI trigger often pull request/push; exact policy depends repo.
- Workflow checks out code, sets runtime, installs dependencies and runs tests/lint/static/security checks.
- CI uses controlled test data/secrets, not production credentials.
- A required status check can block merge when tests fail.
- CI should be deterministic/reasonably fast; expensive full data validations belong in staged integration environments.
- Passing CI proves the checks passed, not that production deployment is safe forever.

Teacher demonstration / case

```
PR -> CI workflow -> tests/quality/static checks -> status
FAIL -> repair PR
PASS -> review/merge allowed
```

Read it like English

- CI is a pre-merge confidence gate and feedback loop.
- Deployment/CD can be a separate later workflow.

Expected check

CI concern	rule
trigger	PR/push
inputs	controlled code/test data
failure	non-zero job/status
secrets	scoped test secret if truly needed

Common beginner traps

- Running production destructive load in PR CI.
- Allowing failing required checks to merge.
- CI only checks syntax.

Now you do a similar task

Design a CI pipeline for one earlier project: exact commands, data fixture, failure behavior and artifacts.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

What important class of problem can pass CI but fail in production?

Answer in your own words before continuing.

DE16-05 - GitHub Actions for data projects

What you will be able to do

Read/write a GitHub Actions workflow for Python data tests without leaking secrets or granting unnecessary permissions.

Why this matters

Workflow YAML is executable automation tied to repository events; unsafe permissions/actions/secrets can affect supply-chain security.

Picture it this way

A CI workflow is a robot account: give it only the tools/permissions needed for the inspection job.

Start from zero

- Workflow has triggers, jobs, runners and steps.
- Pin/trust actions and review third-party actions before giving secrets.
- Set minimal permissions rather than broad write permissions by default when possible.
- Use repository/environment secrets only when required and never echo them.
- Cache dependencies carefully but correctness first.
- Upload test reports/artifacts that help debugging without sensitive data.

Teacher demonstration / case

```
# Inspect supplied .github/workflows/ci.yml
# Typical steps:
- checkout
- setup Python
- install test deps
- run pytest
- run offline quality/security validators
```

Read it like English

- The supplied workflow is an inspectable example; local tests should match CI commands.

Expected check

workflow area	proof
permissions	minimal
secrets	none unless needed
test command	same locally/CI
failure	job red/non-zero

Common beginner traps

- Copying an action from random repo without review.
- Printing secrets for debug.
- Giving contents:write/admin-like permissions to simple test job.

Now you do a similar task

Review the supplied ci.yml line-by-line and write what each step can access/change. Remove any permission you cannot justify.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why is a CI runner with repository write permission more sensitive than a local test command?

Answer in your own words before continuing.

DE16-06 - Dev, test and prod environments

What you will be able to do

Separate dev/test/prod data, config, credentials and change privileges while keeping code promotion reproducible.

Why this matters

Environment separation limits blast radius and makes testing realistic without letting experiments modify production.

Picture it this way

Dev is the workshop, test is the rehearsal stage, prod is the live show; same script, different controlled settings/data/access.

Start from zero

- Code artifact/version should be promoted; environment config/connection endpoints differ.
- Dev may use synthetic/sample data and flexible developer write.
- Test uses production-like schemas/integration with isolated targets.
- Prod restricts writes/deploy to workload identities/operators.
- Never point dev/test accidentally at prod through a copied DATABASE_URL.
- Config files in supplied project demonstrate environment-specific non-secret values; secrets remain separate.

Teacher demonstration / case

```
config/dev.json
config/test.json
config/prod.json
# Same src/pipeline.py reads selected environment config.
# Secret credentials are NOT stored in these JSON files.
```

Read it like English

- Separation enables testing the same code against safer targets.
- Environment variable/approved deployment parameter chooses config; hardcoded prod endpoint is unsafe.

Expected check

env	main purpose
dev	fast safe iteration
test	integration/release validation
prod	live controlled operations

Common beginner traps

- Using production data/credentials in local unit tests.
- Different unmerged code per environment.
- Environment selected by manually editing source.

Now you do a similar task

Run config_probe.py or inspect config assets. Prove dev/prod targets differ while transformation code is identical.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why is copying production credentials into dev worse than simply having a separate dev database?

Answer in your own words before continuing.

DE16-07 - Deployment and rollback

What you will be able to do

Deploy a known version with validation and define rollback/roll-forward behavior for code plus data/schema changes.

Why this matters

Code rollback can be simple; data migrations/writes may be irreversible without an explicit recovery plan.

Picture it this way

Changing a recipe can be rolled back by using yesterday’s recipe, but if today’s recipe already changed every stored ingredient, data recovery is separate.

Start from zero

- Deploy immutable/versioned artifact/commit, not "whatever is on laptop".
- Pre-deploy checks + post-deploy smoke/quality controls.
- Record release version/config/schema changes.
- Rollback code when compatible; roll-forward may be safer for irreversible migrations.
- Database/table schema/data migration needs backup/version/replay plan.
- Canary/staged deployment reduces blast radius for high-risk changes.

Teacher demonstration / case

```
release v1 -> deploy -> smoke/control totals
if failure:
  stop further runs
  assess data impact
  rollback code OR roll forward fix
  restore/replay affected data if necessary
  validate
```

Read it like English

- Recovery decision depends on whether side effects already committed.
- Runbook should distinguish code deployment failure from data corruption.

Expected check

change	rollback concern
stateless code only	usually easier
schema migration	compatibility/data migration
published data rewrite	restore/replay needed
credential change	rotation/connection update

Common beginner traps

- "git revert" assumed to undo database data.
- No post-deploy validation.
- Deploying directly from uncommitted local code.

Now you do a similar task

Use P04 deployment rollback exercise. Write a failure timeline where code v2 corrupts one partition and define containment + data recovery + redeploy validation.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why can rolling back application code fail to restore correct data?

Answer in your own words before continuing.

DE16-08 - Secrets and credential safety

What you will be able to do

Prevent, detect and respond to secret exposure: keep credentials out of code/Git/logs and rotate compromised values.

Why this matters

Credentials grant access; once exposed in Git/log/screenshot, you must assume someone could copy them even if you delete the latest file.

Picture it this way

A leaked key is like a photographed house key: erasing the photo from your phone does not make the copied key invalid.

Start from zero

- Secrets include passwords, API tokens, private keys, SAS URLs/credentials.
- Use secret managers/runtime environment/connection mechanisms; repositories store safe names/templates.
- gitignore prevents new untracked files, not historical committed secrets.
- Secret scanning can detect patterns but is not perfect.
- Incident first action for real leaked secret: revoke/rotate and assess scope/logs, then clean repository/history as appropriate.
- Never use real secrets in a training leak exercise; use obvious fake tokens.

Teacher demonstration / case

```
python G05_SECRET_SCAN.py
# Scan the supplied synthetic project/fake secret patterns.
# If a REAL secret incident: rotate/revoke FIRST.
```

Read it like English

- The lab uses fake values so security training does not create actual risk.
- A clean scan is one control, not proof no credential exists anywhere.

Expected check

stage	action
prevent	secret store + gitignore + review
detect	scanner + code review/log policy
respond	revoke/rotate -> assess -> clean/remediate
verify	old credential no longer works

Common beginner traps

- Deleting secret from latest commit and doing nothing else.
- Posting real token to test scanner.
- Logging connection strings with passwords.

Now you do a similar task

Run secret scanner on fake sample and write a five-step response for a real production token accidentally pushed to public Git.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

What should happen before you spend time rewriting Git history after a real credential leak?

Answer in your own words before continuing.

DE16-09 - RBAC and least privilege

What you will be able to do

Design RBAC permissions by role/action/resource and prove least privilege for pipelines/readers/operators.

Why this matters

Over-broad access increases blast radius of bugs and credential compromise.

Picture it this way

RBAC is assigning key rings by job role: the reporter gets reading-room key, loader gets receiving-door key, not master building key.

Start from zero

- RBAC maps roles/identities to permitted actions/resources.
- Separate data read, data write, deployment/admin, secret access.
- Workload identity should get only source/target resources needed.
- Human emergency/admin access can be separate, audited and time-bound.
- Review inherited/group privileges and stale memberships.
- Test denied operations as well as allowed operations when possible.

Teacher demonstration / case

```
# Inspect G06_RBAC.json
# For each role answer:
# resource scope? read/write/admin? secret access? justification?
```

Read it like English

- JSON asset makes privilege review executable/offline.
- A role matrix is stronger when every grant maps to a pipeline responsibility.

Expected check

role	minimum tendency
BI reader	serving SELECT/read
pipeline loader	specific source read + target write
developer	dev only
prod operator	deploy/incident scopes, not all data by default

Common beginner traps

- Granting admin because permission errors are annoying.
- One shared service account for every pipeline.
- Checking allowed access but never denied access.

Now you do a similar task

Review/repair the supplied RBAC asset to remove unnecessary permissions. Write one allowed and one denied test per role.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

How does least privilege reduce the impact of a buggy pipeline, not only hackers?

Answer in your own words before continuing.

DE16-10 - PII, encryption and masking

What you will be able to do

Classify PII, minimize collection, protect it in transit/at rest and mask/tokenize where full values are unnecessary.

Why this matters

Security starts by not storing/accessing sensitive data unnecessarily. Encryption alone does not make unrestricted plaintext use safe.

Picture it this way

If a report only needs last four digits, do not give every analyst the full identity document and rely on a locked cabinet.

Start from zero

- PII/sensitive classes depend on jurisdiction/policy; identify direct/quasi identifiers and business sensitivity.
- Data minimization: collect/retain only needed fields and duration.
- Encryption in transit protects network path; at rest protects stored media/files depending system.
- Mask/tokenize/hash have different reversibility/use cases; plain unsalted hash may be re-identifiable for low-entropy values.
- Restrict raw PII layer; provide masked/aggregated serving views.
- Logs/test fixtures must avoid leaking real PII.

Teacher demonstration / case

```
python G07_MASK_PII.py
# Synthetic data only: compare raw field vs masked output.
# Keep raw access restricted; downstream uses minimum necessary representation.
```

Read it like English

- Masking changes visibility, not necessarily cryptographic secrecy.
- The correct technique depends on whether later exact joins/recovery are required.

Expected check

need	possible approach
display last 4	mask
stable join without plaintext	tokenization/keyed hash with design
recover original	encryption/token vault
analytics aggregate	drop/minimize identifier

Common beginner traps

- Using MD5/SHA of phone/email and calling it anonymous.
- Copying production PII into dev tests.
- Giving masked table and raw-table read to same broad role.

Now you do a similar task

Classify fields in a customer dataset and propose raw/Silver/Gold exposure, retention and masking. Run fake masking lab.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why is hashing an email address not automatically anonymization?

Answer in your own words before continuing.

DE16-11 - Metadata, lineage and governance

What you will be able to do

Use metadata, lineage and governance to answer ownership, meaning, origin, impact and policy questions.

Why this matters

As systems grow, engineers need to know who owns a dataset, what it means and what downstream breaks before changing it.

Picture it this way

Metadata is the catalog card; lineage is the route map; governance is the agreed rules and accountability for using the collection.

Start from zero

- Technical metadata: schema, type, partitions, size, versions.
- Business metadata: definition, owner, SLA, sensitivity/classification.
- Operational metadata: run IDs, freshness, quality metrics, failures.
- Lineage connects sources -> transformations -> outputs; can be dataset/table/column level.
- Impact analysis walks downstream dependencies before breaking changes.
- Governance defines ownership/access/retention/quality/change processes, not merely a catalog tool.

Teacher demonstration / case

```
# Inspect G08_LINEAGE.json
# Trace: source -> transform -> serving
# Ask: if source.amount changes type, which downstream assets/tests/owners are impacted?
```

Read it like English

- The JSON lineage graph supports offline impact reasoning.
- A graph without owners/contracts is incomplete governance.

Expected check

question	metadata
what does it mean?	business definition
who owns?	owner/domain
where came from?	lineage/source/run
who breaks if changed?	downstream lineage
sensitive?	classification/policy

Common beginner traps

- Cataloging tables but no owner.
- Lineage only as picture not tied to versions.
- Governance as approvals with no engineering automation.

Now you do a similar task

Perform impact analysis on one source field change using lineage asset. List affected nodes, tests, owners and rollout order.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why is schema metadata alone insufficient to tell whether a column change is safe?

Answer in your own words before continuing.

DE16-12 - Observability for data pipelines

What you will be able to do

Design observability across logs, metrics, traces/run lineage and data-quality/freshness signals with actionable alerts.

Why this matters

A pipeline can be running but wrong, late or incomplete. Operations need both software and data signals.

Picture it this way

Observability is the dashboard showing engine temperature plus product counts/quality, not only whether the machine power light is on.

Start from zero

- Logs provide event/context/error detail with run/task IDs.
- Metrics quantify duration, throughput, rejects, freshness, lag, retries, resource use.
- Distributed trace/run lineage links steps/services to one run where tooling supports.
- Data observability includes schema/volume/key/measure/freshness anomalies.
- Alerts need owner, threshold/SLA and actionable runbook link.
- Avoid alert fatigue: page on customer/SLA-impacting conditions, ticket/monitor lower severity.

Teacher demonstration / case

```
python G09_OBSERVABILITY.py
# Inspect normal/bad metric output and decide which signals should alert.
```

Read it like English

- The lab creates synthetic operational/data metrics.
- Your task is to connect metric -> symptom -> likely cause -> runbook.

Expected check

signal	example
freshness_lag	source/pipeline late
source vs target rows	loss/rejection
retry/error rate	dependency instability
duration	performance/capacity regression

Common beginner traps

- Alert on every single retry.
- Only infrastructure CPU metrics, no data quality.
- Logs without run IDs.

Now you do a similar task

Run observability lab and design three alerts with threshold, severity, owner and first runbook check.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

How can you distinguish "pipeline produced no rows because source was empty" from "pipeline silently lost all rows"?

Answer in your own words before continuing.

DE16-13 - Incidents, runbooks and root cause

What you will be able to do

Respond to incidents by containing impact, preserving evidence, using runbooks and writing root-cause/action follow-up instead of blame.

Why this matters

Production failures are inevitable; disciplined response reduces damage and repeat incidents.

Picture it this way

A runbook is the emergency checklist; RCA is the engineering investigation after the fire is controlled.

Start from zero

- Incident flow: detect -> assess severity/data impact -> contain -> diagnose -> recover -> validate -> communicate -> learn.
- Preserve logs/run IDs/input/output versions before deleting/retrying evidence.
- Runbook gives known checks/recovery for common symptoms, but operators still reason about current context.
- Root cause distinguishes trigger, contributing factors and why controls failed to catch/prevent it.
- Corrective actions include code/test/monitor/process/security improvements with owners/dates.
- Postmortem should be factual and learning-oriented, not person-blaming.

Teacher demonstration / case

```
# RUNBOOK.md structure
Symptom
Impact/SLA
First evidence (run_id, logs, counts)
Containment
Safe retry/rollback/replay steps
Validation
Escalation/owner
Known risks
```

Read it like English

- The supplied project includes RUNBOOK and normal/bad evidence.
- A safe retry step must mention idempotency/state.

Expected check

incident phase	goal
contain	stop further bad writes
diagnose	evidence-based cause
recover	restore/replay safely
validate	controls + consumer impact
RCA	prevent recurrence

Common beginner traps

- Deleting failed output/logs before investigation.
- Retrying repeatedly while corruption continues.
- RCA = "engineer made mistake".

Now you do a similar task

Use P09 incident RCA on a broken-quality deployment. Write containment, evidence, root cause and three preventive actions.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why should containment often happen before you know the exact root cause?

Answer in your own words before continuing.

DE16-14 - Terraform and IaC foundations

What you will be able to do

Understand Infrastructure as Code and review a small Terraform configuration/state/plan workflow without using it as copy-paste cloud magic.

Why this matters

Manual infrastructure changes drift and are hard to reproduce/audit. IaC defines desired resources/config in versioned code.

Picture it this way

Terraform is a blueprint plus change calculator: configuration says desired building, state records known built resources, plan previews differences.

Start from zero

- Terraform HCL declares providers/resources/data/variables/outputs.
- terraform init prepares provider/plugins/backend; plan previews changes; apply changes infrastructure.
- Terraform state maps configuration to real resources and may contain sensitive metadata; protect remote state appropriately.
- Never commit local tfstate with secrets; use approved backend/state locking in team environments.
- Review plan for destructive replacements/deletes before apply.
- IaC manages infrastructure configuration, not application/data correctness by itself.

Teacher demonstration / case

```
# Inspect practice_project/terraform/
# Offline validator checks HCL structure.
# Conceptual workflow:
terraform fmt -check
terraform validate
terraform plan
# Review plan BEFORE apply
```

Read it like English

- The course intentionally uses offline/static Terraform review unless you have a safe cloud sandbox.
- No cloud apply is required to understand IaC foundations.

Expected check

artifact	meaning
.tf	desired config
state	known resource mapping/sensitive
plan	proposed diff
apply	perform diff

Common beginner traps

- terraform apply copied from tutorial against real cloud account.
- Committing tfstate.
- Approving plan without noticing resource destroy/recreate.

Now you do a similar task

Run offline HCL validator and review a sample plan conceptually: identify one safe add and one destructive replacement you would stop.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why is Terraform state a security/operational asset rather than just a disposable cache file?

Answer in your own words before continuing.